204252 Data Structures and Analysis

Chapter 11: Graphs (Part 4) - Shortest Paths

Lecturer

August 31, 2026

Computer Science Department

Outline

Introduction to Shortest Paths

Unweighted Shortest Paths (BFS)

Dijkstra's Algorithm (Distance Only)

Shortest Path Reconstruction (Backtracking)

Introduction to Shortest Paths

The Shortest-Path Problem

Given a weighted graph $G = (V, E)$ and a source vertex s, find the shortest path from s to all other vertices in G.

- **Single-Source Shortest-Path**: Find the shortest path from s to every vertex v.
- **Edge Weight Types**:
  - **Unweighted**: All edges have weight 1.
  - **Non-negative weights**: All weights $w(u, v) \geq 0$ (handled by Dijkstra's).
  - **Negative weights**: Some weights $w(u, v) < 0$ (requires Bellman-Ford).
- **Negative Cycles**: If a graph contains a cycle whose sum of weights is negative, shortest paths are undefined (we can traverse the cycle infinitely to get distance $-\infty$).

Unweighted Shortest Paths (BFS)

Unweighted Shortest Path Algorithm

- If a graph is unweighted, we can use a modified **Breadth-First Search (BFS)**.
- Because BFS explores vertices level-by-level, the first time we see a vertex, we have found the shortest path to it from the source.
- We track distance and path (parent) for each vertex.
- **Complexity**: $\mathcal{O}(|V| + |E|)$ with Adjacency List.

Unweighted Shortest Path C++

```
void Graph::unweightedShortestPath(int s) {
    vector<int> dist(numVertices, INT_MAX);
    vector<int> path(numVertices, -1);
    queue<int> q;

    dist[s] = 0;
    q.push(s);

    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adjLists[u]) {
            if (dist[v] == INT_MAX) { // v is unvisited
                dist[v] = dist[u] + 1;
                path[v] = u;
                q.push(v);
            }
        }
    }
}
```

Dijkstra's Algorithm (Distance Only)

Dijkstra's Algorithm - Overview

- **Greedy Strategy**: Maintain a set of vertices whose shortest distance from the source is already known.
- At each step, pick an unknown vertex u with the minimum tentative distance d_u.
- Mark u as **known**.
- **Relax** all outgoing edges (u, v) from u:

$$\text{if } d_u + w(u, v) < d_v \implies d_v = d_u + w(u, v)$$

- **Requirement**: Edge weights must be **non-negative** ($w(u, v) \geq 0$).

Dijkstra's C++ Pseudocode (Distance Only)

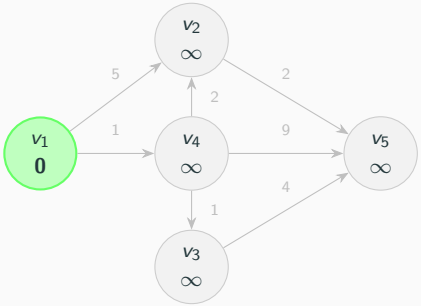
```cpp
void Graph::DijkstraDistanceOnly(int s) {
    vector<int> dist(numVertices, INT_MAX);
    vector<bool> known(numVertices, false);
    dist[s] = 0;

    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;
    pq.push({0, s});

    while (!pq.empty()) {
        int u = pq.top().second; pq.pop();
        if (known[u]) continue;
        known[u] = true;

        for (auto edge : adjLists[u]) {
            int v = edge.dest, w = edge.weight;
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
}
```

Dijkstra (Distance Only) Step-by-Step (Initial State (Source = v1))

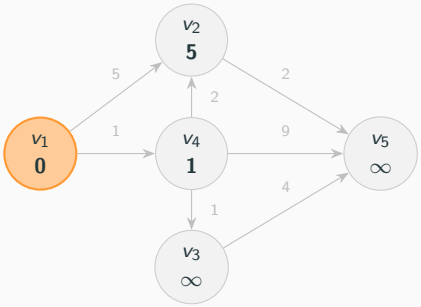


Dijkstra Table (Distance Only):

Node	Known	Dist d_v
v_1	T	0
v_2	F	∞
v_3	F	∞
v_4	F	∞
v_5	F	∞

Shortest Distance to v5: ∞

Dijkstra (Distance Only) Step-by-Step (Step 1: Process v1)

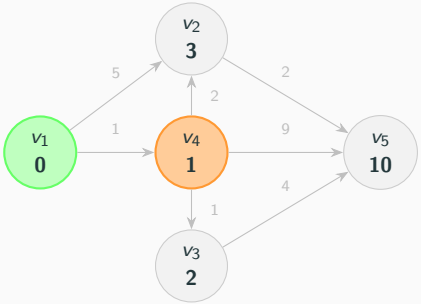


Dijkstra Table (Distance Only):

Node	Known	Dist d_v
v_1	T	0
v_2	F	5
v_3	F	∞
v_4	F	1
v_5	F	∞

Shortest Distance to v5: ∞

Dijkstra (Distance Only) Step-by-Step (Step 2: Process v4 (Relax edges to v2, v3, v5))

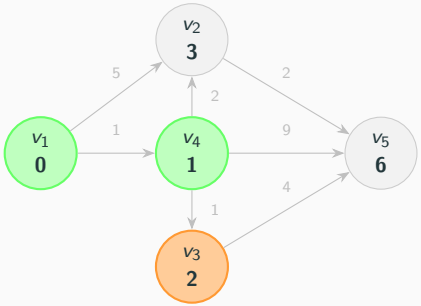


Dijkstra Table (Distance Only):

Node	Known	Dist d_v
v_1	T	0
v_2	F	3
v_3	F	2
v_4	T	1
v_5	F	10

Shortest Distance to v5: **10**

Dijkstra (Distance Only) Step-by-Step (Step 3: Process v3 (Relax edge to v5))

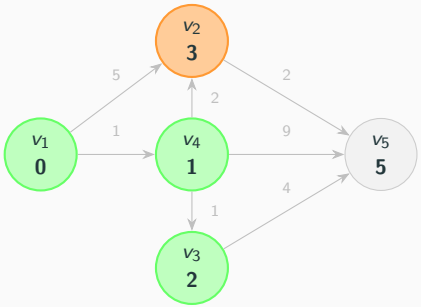


Dijkstra Table (Distance Only):

Node	Known	Dist d_v
v_1	T	0
v_2	F	3
v_3	T	2
v_4	T	1
v_5	F	6

Shortest Distance to v5: **6**

Dijkstra (Distance Only) Step-by-Step (Step 4: Process v2 (Relax edge to v5))

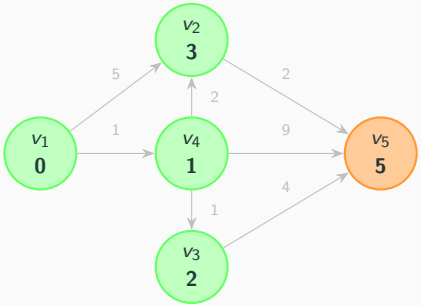


Dijkstra Table (Distance Only):

Node	Known	Dist d_v
v_1	T	0
v_2	T	3
v_3	T	2
v_4	T	1
v_5	F	5

Shortest Distance to v5: **5**

Dijkstra (Distance Only) Step-by-Step (Step 5: Process v5 (Destination reached))



Dijkstra Table (Distance Only):

Node	Known	Dist d_v
v_1	T	0
v_2	T	3
v_3	T	2
v_4	T	1
v_5	T	5

Shortest Distance to v5: **5**

Shortest Path Reconstruction (Backtracking)

Reconstructing the Shortest Path

- Finding the shortest path distance is useful, but we also want the actual sequence of vertices (the path).
- We can record the path by keeping track of the **predecessor** (parent) of each vertex:
  - When we relax an edge (u, v) and update $d_v = d_u + w(u, v)$, we also set $p_v = u$.
- Once the algorithm finishes, we can reconstruct the path by **backtracking** from the destination v back to the source s:
  - $v \to p_v \to p_{p_v} \to \cdots \to s$
- Let's trace the same example graph again, this time keeping track of the parent pointer p_v.

Dijkstra's C++ Pseudocode (With Path Tracking)

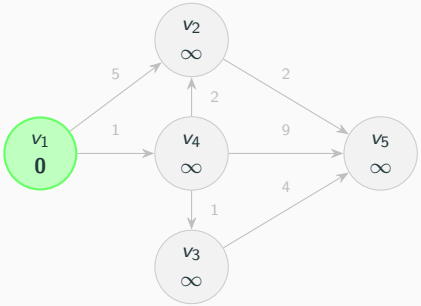
```cpp
void Graph::Dijkstra(int s) {
    vector<int> dist(numVertices, INT_MAX);
    vector<int> parent(numVertices, -1); // Tracks predecessor
    vector<bool> known(numVertices, false);
    dist[s] = 0;

    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;
    pq.push({0, s});

    while (!pq.empty()) {
        int u = pq.top().second; pq.pop();
        if (known[u]) continue;
        known[u] = true;

        for (auto edge : adjLists[u]) {
            int v = edge.dest, w = edge.weight;
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                parent[v] = u; // Record the predecessor u
                pq.push({dist[v], v});
            }
        }
    }
}
```

Dijkstra (With Path Tracking) Step-by-Step (Initial State (Source = v1))

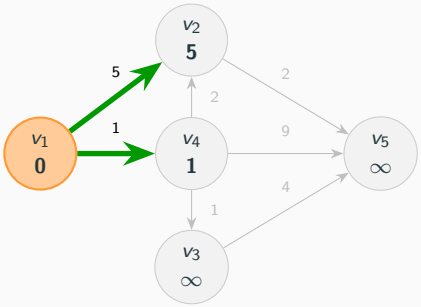


Dijkstra Table with Parents:

Node	Known	Dist d_v	Parent p_v
v_1	T	0	-
v_2	F	∞	-
v_3	F	∞	-
v_4	F	∞	-
v_5	F	∞	-

Parent pointers are updated whenever a cheaper edge relaxation occurs.

Dijkstra (With Path Tracking) Step-by-Step (Step 1: Process v1)

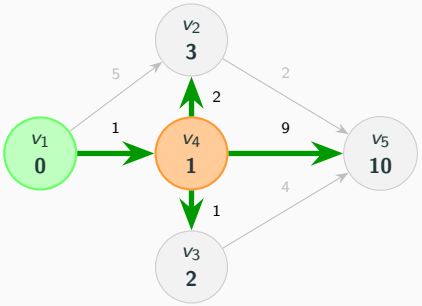


Dijkstra Table with Parents:

Node	Known	Dist d_v	Parent p_v
v_1	T	0	-
v_2	F	5	v1
v_3	F	∞	-
v_4	F	1	v1
v_5	F	∞	-

Parent pointers are updated whenever a cheaper edge relaxation occurs.

Dijkstra (With Path Tracking) Step-by-Step (Step 2: Process v4 (Relax edges to v2, v3, v5))

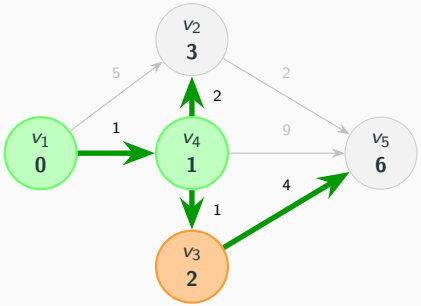


Dijkstra Table with Parents:

Node	Known	Dist d_v	Parent p_v
v_1	T	0	-
v_2	F	3	v4
v_3	F	2	v4
v_4	T	1	v1
v_5	F	10	v4

Parent pointers are updated whenever a cheaper edge relaxation occurs.

Dijkstra (With Path Tracking) Step-by-Step (Step 3: Process v3 (Relax edge to v5))

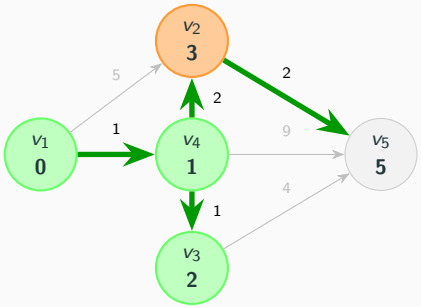


Dijkstra Table with Parents:

Node	Known	Dist d_v	Parent p_v
v_1	T	0	-
v_2	F	3	v4
v_3	T	2	v4
v_4	T	1	v1
v_5	F	6	v3

Parent pointers are updated whenever a cheaper edge relaxation occurs.

Dijkstra (With Path Tracking) Step-by-Step (Step 4: Process v2 (Relax edge to v5))

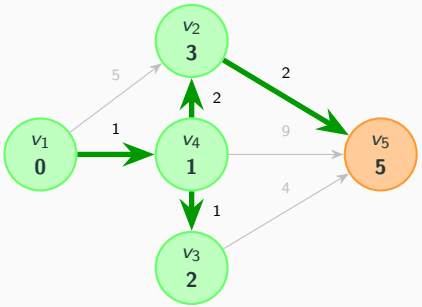


Dijkstra Table with Parents:

Node	Known	Dist d_v	Parent p_v
v_1	T	0	-
v_2	T	3	v4
v_3	T	2	v4
v_4	T	1	v1
v_5	F	5	v2

Parent pointers are updated whenever a cheaper edge relaxation occurs.

Dijkstra (With Path Tracking) Step-by-Step (Step 5: Process v5 (Destination reached))

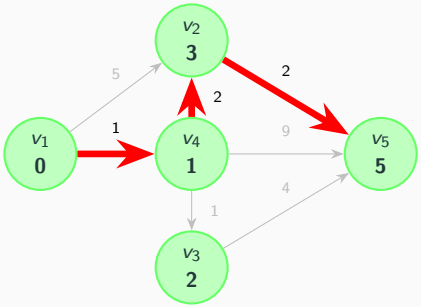


Dijkstra Table with Parents:

Node	Known	Dist d_v	Parent p_v
v_1	T	0	-
v_2	T	3	v4
v_3	T	2	v4
v_4	T	1	v1
v_5	T	5	v2

Parent pointers are updated whenever a cheaper edge relaxation occurs.

Backtracking Example (Finding Path to v5)



Backtracking from v_5 to v_1:

- Start at target v_5 (parent$[v_5] = v_2$)
- Move to v_2 (parent$[v_2] = v_4$)
- Move to v_4 (parent$[v_4] = v_1$)
- Move to v_1 (no parent $\rightarrow$ source reached)

Reconstructed Path (Reversed):

$v_1 \rightarrow v_4 \rightarrow v_2 \rightarrow v_5$

Shortest Path Cost: **5**

Path Reconstruction C++ Code

- We can print the path in correct order (from source to destination) using a simple recursive function:

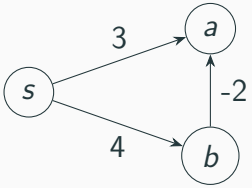
```cpp
void printPath(int v, const vector<int>& parent) {
    if (parent[v] != -1) {
        printPath(parent[v], parent);
        cout << " -> ";
    }
    cout << v;
}
```

- Works for both weighted (Dijkstra's) and unweighted (BFS) algorithms.
- **Complexity**: Time $\mathcal{O}(\text{path length}) \leq \mathcal{O}(|V|)$, Auxiliary Space $\mathcal{O}(|V|)$ stack frames.

Why Dijkstra Fails on Negative Weights

- Dijkstra's algorithm relies on the fact that once a vertex is marked known, its shortest path cannot be improved.
- This is because any path to an unknown vertex must pass through a vertex that is already at least as far away.
- If negative weights exist, this assumption is violated: a path that looks longer initially can become shorter by traversing a negative edge later.
- Once a vertex is marked known, Dijkstra's algorithm never updates its distance again, leading to incorrect results.

Negative Weights Counterexample



Dijkstra's Execution:

1. Start at s. Tentative: $d_a = 3, d_b = 4$.
2. Pop a (dist 3). Mark a **known**.
3. Pop b (dist 4). Mark b **known**.
4. Relax edge (b, a):
   $d_b + w(b, a) = 4 - 2 = 2 < 3$.
5. But a is already marked known and finished! Dijkstra's fails to find the actual shortest path $s \to b \to a$ (cost 2).